

RUNTIME FOR EXECUTION & EXCHANGE ORCHESTRATION

REXO

Architectural Constitution

Complete specification of the core, capabilities, execution, Economy Engine, memory, packs, marketplace, governance, Developer Experience, visual Studio and Platform Factory.



Ylanna Almeida

Founder and originator of the project - original vision and product direction

VERSION	3.0 - EN edition
DATE	July 2026
STATUS	Core v1 frozen - Phase 0 in progress
TECHNICAL WRITING	In collaboration with AI agents

GOVERNANCE

Document Control

Field	Definition
Title	REXO - Architectural Constitution
Version	3.0 - English edition
Vision	Ylanna Almeida - original vision, product direction and founding of the project
Collaboration	Architecture and technical writing developed with OpenAI Codex
State	Core v1 frozen; Public Foundation v0.0.1 in incremental execution
Purpose	Define the platform's invariants, components, contracts and evolution model
Horizon	Architecture prepared for long-term evolution and hundreds of logical specialists
First distribution	REXO Education
Source of truth	This document, approved ADRs, versioned contracts and published policies

Interpretation rule. The original vision belongs to Ylanna Almeida. The discovery conversation remains as historical context. This document consolidates and supersedes exploratory decisions in case of conflict.

Contents

Document Control

2

1. Mandate and Vision

8

1.1 Intended Outcome

8

1.2 Non-Goals

8

2. Architectural Principles

8

2.1 The Right Metaphor

9

3. Logical Architecture

11

3.1 Control Plane

11

3.2 Execution Plane

11

3.3 Knowledge Plane

11

3.4 Quality and Governance Plane

12

4. Capability Model

12

4.1 Capability Specification

12

4.2 Capability Implementation

12

4.3 Capability Resolver

12

4.4 Taxonomy

13

5. Agents, Skills and Workflows

13

5.1 Agents

13

5.2 Skills

13

5.3 Workflows

13

5.4 Selection Rule

14

6. Execution Flow

14

6.1 Task Envelope

14

6.2 Scaling to Hundreds of Agents

15

7. Memory and Knowledge

15

7.1 Context Service

16

7.2 Knowledge Promotion

16

7.3 Knowledge Base

16

8. Tool Gateway and MCP

16

8.1 Gateway Responsibilities

16

8.2 Effect Classes

17

8.3 ComfyUI

17

9. Packs, Plugins and Marketplace	19
9.1 Package Contents	19
9.2 Marketplace Security	19
9.3 Package Manager	19
10. Apps and Distribution Builder	19
10.1 Future Examples	20
11. REXO Education	20
11.1 Composition	20
11.2 Full Pipeline	20
11.3 Educational Quality Gates	21
12. Quality and Evaluation	23
12.1 Definition of Done	23
13. Security, Isolation and Governance	23
13.1 Trust Boundaries	23
13.2 Mandatory Rules	24
13.3 Human-in-the-loop	24
14. Versioning and Reproducibility	24
14.1 Run Manifest	24
15. Economy Engine, Observability and Operations	24
15.1 Economy Engine	25
15.2 Economic Decision Sequence	25
15.3 Economic Rules	25
15.4 Telemetry	25
15.5 Budget Manager	26
15.6 Initial SLOs	26
16. Continuous Evolution	26
16.1 Improvements That Can Be Proposed	26
16.2 Protected Components	26
16.3 Promotion	27
17. Directory Structure	29
17.1 Generated Catalog	30
18. Fundamental Contracts	30
18.1 Extension Without Core Changes	31
19. Risks and Controls	31

- 20. Future Implementation Strategy 31
 - 20.1 Criteria for Extracting Microservices 32
- 21. How We Will Work 32
 - 21.1 Decision Process 32
 - 21.2 Rules for the Chief AI Architect 32
 - 21.3 Rules for Future Deliveries 32
- 22. Architectural Decisions Approved in This Proposal 33
- 23. Developer & Creation Plane 35
 - 23.1 Single-Format Invariant 35
 - 23.2 Components 35
 - 23.3 Extensibility Rule 36
- 24. REXO SDK 36
 - 24.1 Conceptual APIs 36
 - 24.2 Language Strategy 36
 - 24.3 Conformance Test Kit 37
- 25. REXO CLI, Scaffolding and Generators 37
 - 25.1 Target Interface 37
 - 25.2 Pack Creation Journey 37
 - 25.3 Generators 37
- 26. REXO Studio and REXO Canvas 38
 - 26.1 Workflow Canvas 38
 - 26.2 Domain Canvas 38
 - 26.3 Round-trip Fidelity 38
- 27. Runtime Marketplace and Installation Lifecycle 38
 - 27.1 Transactional Installation 39
 - 27.2 Update and Removal 39
 - 27.3 Runtime Profiles 39
- 28. REXO Creator 39
 - 28.1 Platform Intent Specification 40
 - 28.2 Outputs 40
 - 28.3 Limits 40
 - 28.4 Example 40
- 29. Operating Modes and Activation 41
 - 29.1 How to Activate With Me Right Now 41
 - 29.2 Activation Confirmation 42

29.3 Future CLI Interface42

29.4 State Machine42

29.5 How I Will Operate on the Project42

Appendix A. Glossary44

Appendix B. Approval Checklist44

Next Step46



PART

Foundation

Mandate, vision, non-goals and the principles that govern every decision that follows.

01



FOUNDATION

1. Mandate and Vision

REXO is an extensible operational platform for composing, executing, governing and distributing artificial intelligence capabilities. Its core provides durable orchestration, capability resolution, secure tool access, governed memory, evaluation, observability and package management.

The platform will not be defined by a fixed set of agents. Agents, workflows, applications and distributions will be built on top of capability contracts. The goal is to allow new domains to be installed without altering the core.

Capability-first. Pack-native. Agent-optional.

1.1 Intended Outcome

- Receive high-level goals and turn them into verifiable plans and final deliverables.
- Dynamically compose specialists, Skills, workflows, tools, models and policies.
- Enable specialized distributions, such as Education, Software Factory, Creative and Marketing.
- Learn in a governed way and reuse validated knowledge across projects.
- Support local, private, hybrid and cloud execution.
- Scale from a single installation to a multi-tenant platform.

1.2 Non-Goals

- Do not build a monolithic chatbot with one giant prompt.
- Do not literally map every corporate role to a permanent process.
- Do not allow unrestricted communication between agents.
- Do not couple workflows to specific vendors, models or MCPs.
- Do not allow direct self-modification of the core, policies or security.
- Do not start with microservices or distributed infrastructure without proven need.

CONSTITUTION

2. Architectural Principles

Principle	Practical consequence
Capability-first	The fundamental unit is a contractual capability; agents are compositions.
Contracts before implementations	Schemas and protocols are defined before concrete components.
Durable by default	Long-running executions survive restarts, failures, pauses and resumptions.
Least privilege	Permissions belong to the task and expire; they do not permanently belong to the agent.
Evidence over confidence	Critical decisions require sources, tests, metrics or verifiable artifacts.
Memory is governed data	Memory has provenance, confidence, scope, validity and retention.
Quality is executable	Quality criteria are evaluators and tests, not adjectives in prompts.

Principle	Practical consequence
Provider independence	The core knows capabilities, not specific vendors.
Reproducibility	Every run records versions, configurations, policies, models, tools and hashes.
Evolution through promotion	Improvements go through sandbox, evals, canary, observation and promotion.

2.1 The Right Metaphor

The term Operating System describes REXO's role as a runtime and application platform for AI. It will not be a traditional operating system kernel. The analogy guides responsibilities but must not produce artificial complexity.

Operating system	REXO
Process	Run or task executed by an ephemeral worker
Scheduler	Task scheduler for priorities, budgets and concurrency
Driver	Capability provider or Tool Gateway adapter
Application	AI App or Application Pack
Package	Provider, Capability, Domain, Policy, Application or Distribution Pack
File system	Artifact Store and Knowledge Plane
Permission	Capability token and execution policy
Distribution	Composition of packs, apps, providers and policies



PART

Architecture

Logical planes, capability model, composition, runtime, memory and integrations.

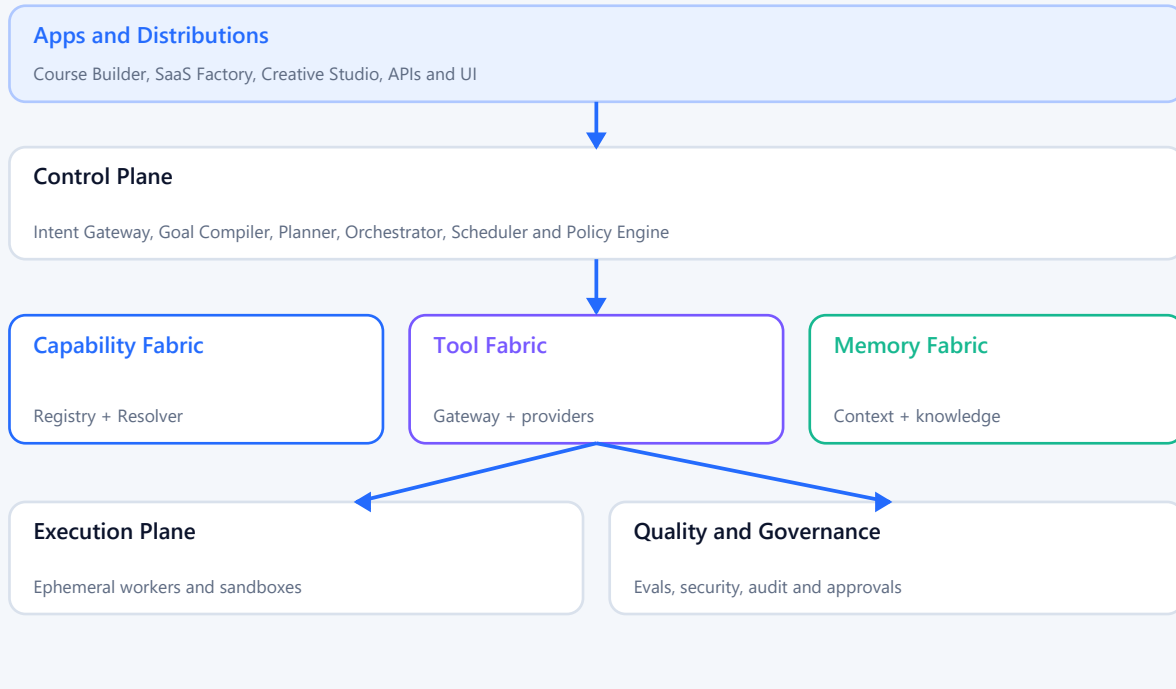
02



SYSTEM VIEW

3. Logical Architecture

REXO — architecture by planes



3.1 Control Plane

- Intent Gateway: normalizes requests coming from UI, API, CLI and automations.
- Goal Compiler: turns natural language into a typed Goal Specification.
- Planner: creates a DAG of tasks, dependencies, gates and acceptance criteria.
- Plan Validator: checks for cycles, permissions, risks, costs and completeness.
- Durable Orchestrator: controls state, retries, timeouts, cancellation and resumption.
- Scheduler: distributes tasks according to priority, budget, capacity and resources.
- Policy Engine: enforces security, approval, retention, compliance and limits.

3.2 Execution Plane

Workers are ephemeral, stateless whenever possible, and run in sandboxes proportional to risk. Each worker receives minimal context, version-pinned Skills, temporary capability tokens, a budget and an output contract.

3.3 Knowledge Plane

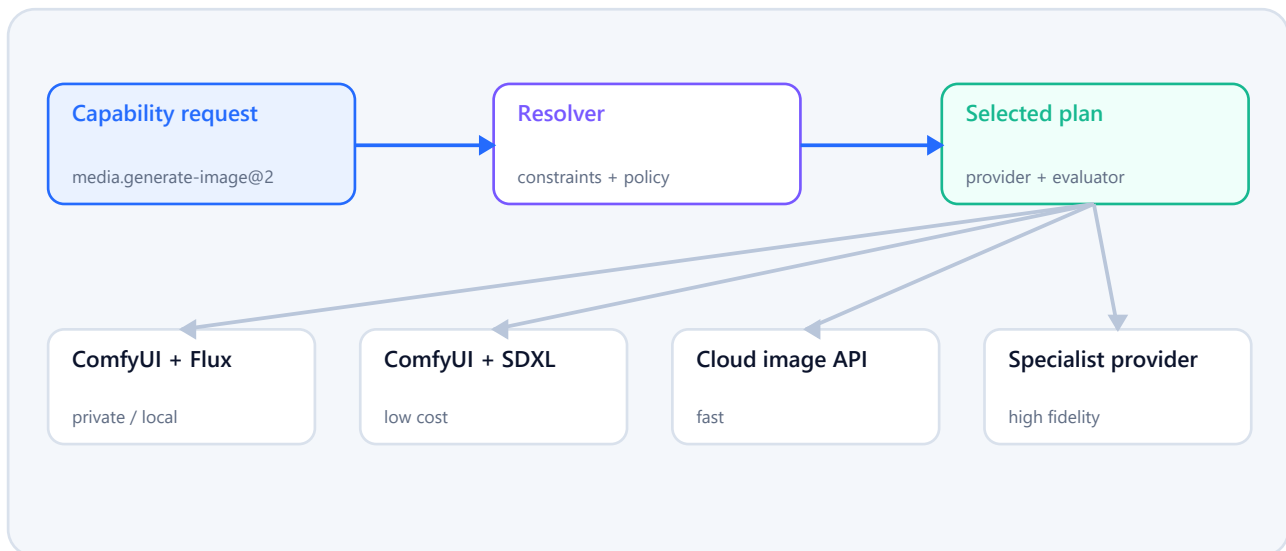
The Knowledge Plane separates the official record, artifacts, search indexes and curated knowledge. Vector indexes are derived and rebuildable; they are never the official source.

3.4 Quality and Governance Plane

Quality is cross-cutting: it validates the plan before execution, checks intermediate outputs, runs regressions and controls the promotion of components.

CONCEPTUAL CORE

4. Capability Model



4.1 Capability Specification

An abstract, versioned, vendor-independent contract. It defines semantics, input and output schemas, possible effects, risks, quality classes, minimum observability and idempotency requirements.

```

kind: CapabilitySpecification
apiVersion: rexo.dev/v1
name: media.generate-image
version: 2.0.0
inputSchema: schemas/generate-image.input.json
outputSchema: schemas/image-artifact.output.json
effects: [artifact.write]
riskClass: medium
qualityProfile: visual-image-v2
  
```

4.2 Capability Implementation

An implementation declares how it satisfies the specification: provider, models, tools, Skills, costs, latency, regions, constraints and evaluators.

4.3 Capability Resolver

- Filters out implementations incompatible with policies, permissions, region or hardware.
- Eliminates options that are unavailable or outside budget.
- Ranks candidates by expected quality, cost, latency, privacy and track record.
- Can run competition between providers when the value of the result justifies it.
- Selects compatible evaluators and produces an auditable Resolution Record.

- Applies fallback only when the contract allows it, and records the substitution.

4.4 Taxonomy

Capabilities will use stable namespaces, such as `education.*`, `media.*`, `software.*`, `knowledge.*`, `quality.*` and `publishing.*`. The taxonomy describes outcome, not technology.

COMPOSITION

5. Agents, Skills and Workflows

5.1 Agents

Agent Definition is a logical role composed of policies, decision-making capabilities, Skill references, memory profile, budgets and delegation criteria. Execution instances do not retain personality or unlimited memory.

Agent contains	Agent does not contain
Goal and decision boundaries	Permanent credentials
Capabilities it may request	Fixed provider implementations
Memory policy	Embedded global workflow
Delegation rules	Duplicated Skill knowledge
Quality profile	Permanent operational state

5.2 Skills

A Skill is a versioned unit of competence. It must have a manifest, instructions, schemas, resources, optional scripts, examples, tests, evals and a changelog.

```
skill/
|-- skill.yaml
|-- instructions.md
|-- input.schema.json
|-- output.schema.json
|-- resources/
|-- scripts/
|-- templates/
|-- examples/
|-- tests/
|-- evals/
`-- CHANGELOG.md
```

5.3 Workflows

A workflow is a durable, versioned DAG. It coordinates capabilities, deterministic tasks, approvals, compensations and evaluations. Skills do not control the global workflow; agents do not replace the orchestrator.

5.4 Selection Rule

Need	Right component
Reusable competence	Skill
Vendor-independent abstract outcome	Capability
Durable, auditable sequence	Workflow
Contextual decision within limits	Agent Definition
External operation	Tool Provider
Complete end-user experience	App

RUNTIME

6. Execution Flow



1. Compile: Convert the goal into a Goal Specification with deliverables, constraints, budget, risk and acceptance criteria.
2. Plan: Produce a typed DAG with dependencies, tasks, gates, retries, timeouts and expected artifacts.
3. Validate: Reject plans that are incomplete, unsafe, cyclical, excessive, or lack verifiable criteria.
4. Resolve: Select capabilities, implementations, Skills, workers, providers and evaluators.
5. Execute: Distribute tasks to ephemeral workers with leases, temporary tokens and minimal context.
6. Evaluate: Run deterministic tests, rubrics, independent review and approvals proportional to risk.
7. Correct: Create targeted tasks for failures; do not restart the entire pipeline unnecessarily.
8. Deliver: Assemble the final package only with approved artifacts and a provenance report.
9. Learn: Generate learning candidates; no promotion happens directly during the run.

6.1 Task Envelope

All inter-agent communication will occur through a Task Envelope or Artifact Contract. Free-form conversation is not the official operating mechanism.

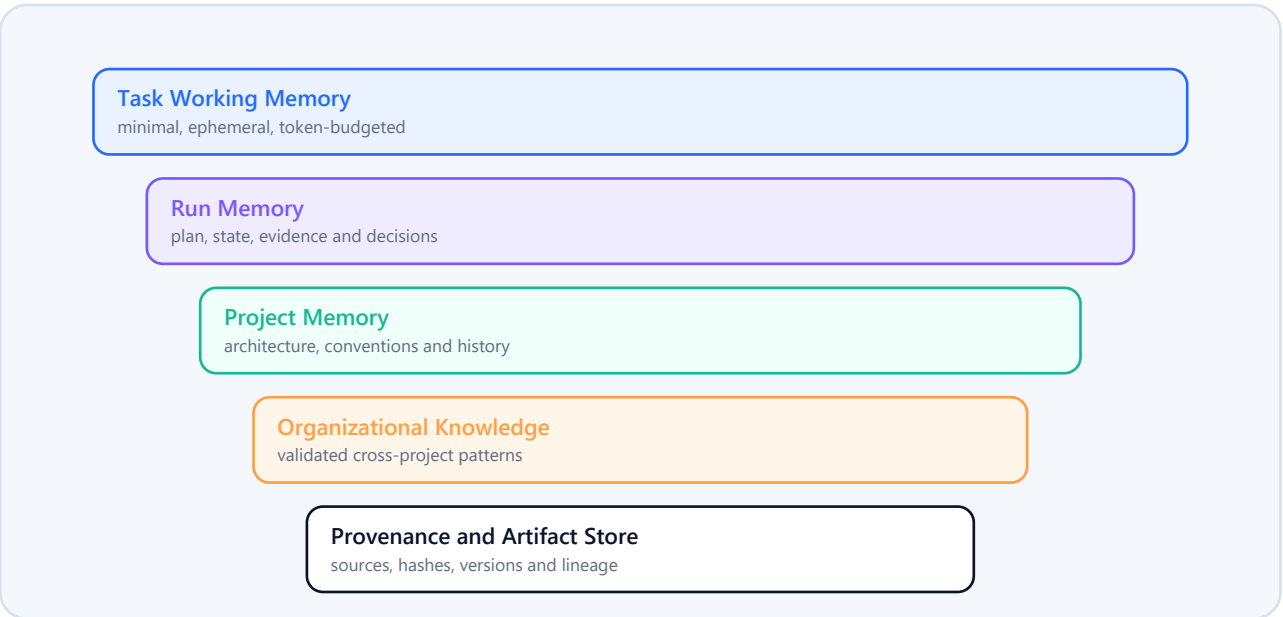
```
taskId
runId
parentTaskId
capability
inputArtifacts
expectedOutputSchema
acceptanceCriteria
skillLock
memoryPolicy
toolPolicy
budget
timeout
retryPolicy
riskClass
traceContext
```

6.2 Scaling to Hundreds of Agents

Hundreds of agents mean hundreds of definitions available in the catalog. The Scheduler only creates the instances that are needed. Backpressure, quotas, leases, priority queues and fan-out limits prevent execution explosion.

MEMORY FABRIC

7. Memory and Knowledge



Layer	Content	Retention	Access
Task working	Minimal context of a task	Ephemeral	Current worker
Run	Plan, state, decisions and evidence	Run lifetime + retention	Run components
Project	Architecture, conventions and history	Long	Project policy
Agent-type	Validated heuristics by type and version	Versioned	Compatible Agent Definition
User preferences	Language, style, limits and preferences	Long and editable	Policy-controlled
Organization	Validated knowledge across projects	Long and curated	Organizational scope

Layer	Content	Retention	Access
Artifact/provenance	Files, hashes, sources and lineage	Per policy	Authorized references

7.1 Context Service

- Receives the task, scope and token budget.
- Queries only authorized memories.
- Prioritizes facts with source, validity and confidence.
- Removes duplication and stale content.
- Produces a Context Manifest recording what was provided to the worker.
- Prevents data from another tenant or project from being included.

7.2 Knowledge Promotion

Observations become candidates. Candidates are deduplicated, classified, verified, evaluated and promoted. Organizational knowledge requires provenance and cannot be written directly by a worker.

7.3 Knowledge Base

The ingestion pipeline preserves the raw document, normalized version, derived chunks, extracted claims, metadata, embeddings and relations. Every factual answer must be able to trace back to the original source.

INTEGRATIONS

8. Tool Gateway and MCP

MCP is a protocol for accessing tools, not the platform's architecture. REXO abstracts MCPs, APIs, CLIs, local applications and cloud services as capability providers.

8.1 Gateway Responsibilities

- Discovery, health checks and provider registration.
- Credential isolation and rotation.
- Temporary capability tokens and least privilege.
- Rate limits, timeouts, circuit breakers and safe retries.
- Normalization of inputs, outputs and errors.
- Auditing of all external effects.
- Approval before destructive, financial, public or irreversible actions.
- Controlled and logged fallback.

8.2 Effect Classes

Class	Example	Typical policy
Read	Query documentation or database	Allowed within scope
Workspace write	Create or edit a file	Allowed within the project
External write	Send a message or update a SaaS	Specific authorization
Publish	Publish a course, site or video	Approval gate
Financial	Purchase or exceptional spend	Mandatory approval
Destructive	Delete data or replace a resource	Approval and backup
Administrative	Permissions, users and credentials	Privileged control

8.3 ComfyUI

ComfyUI will be a multimedia provider. Model selection, workflow, LoRA, VAE, sampler, scheduler, ControlNet, upscaler and VRAM optimization will be separate capabilities. The system will be able to generate alternatives, evaluate results and regenerate only what is needed.



PART

Ecosystem

Packs, distributions, apps and the first reference vertical: REXO Education.

03



ECOSYSTEM

9. Packs, Plugins and Marketplace

Type	Responsibility	Examples
Provider Pack	Integrations and concrete implementations	ComfyUI, OpenAI, Blender, YouTube
Capability Pack	Reusable competencies	Video Production, Research, Documents
Domain Pack	Domain knowledge, policies and evals	Education, Legal, Finance
Policy Pack	Compliance and governance	LGPD, Enterprise Security
Application Pack	Complete end-user experience	Course Builder, SaaS Factory
Distribution Pack	Publishable composition of packs and apps	REXO Education

9.1 Package Contents

- Manifest, publisher, signature, license and version.
- Provided capabilities and required dependencies.
- Skills, workflows, templates, evaluators and Agent Definitions.
- Optional providers and MCP adapters.
- Permissions manifest and data policies.
- Compatibility ranges, migrations and uninstall plan.
- SBOM, changelog, eval baselines and trust tier.

9.2 Marketplace Security

Third-party packages are untrusted software. Installation must resolve dependencies without executing arbitrary hooks on the Control Plane. Packs will be signed, verified, classified by trust and run in a sandbox compatible with their permissions.

9.3 Package Manager

- Installs, updates, disables, removes and rolls back packages.
- Maintains a lockfile per distribution.
- Detects conflicts in capabilities, schemas, policies and versions.
- Runs transactional migrations and supports rollback.
- Generates a resolved catalog, compatibility report and permission diff.

A PLATFORM FOR PLATFORMS

10. Apps and Distribution Builder

Apps are user-facing experiences. Distributions combine Apps, Packs, Providers, Policies, configuration and identity. The Distribution Builder allows building vertical platforms without modifying the REXO Core.

```
distribution:
  name: REXO Education
  coreCompatibility: ">=1.0 <2.0"
  packs:
    - education-domain
    - course-production
    - creative-media
    - publishing
    - learning-analytics
  providers:
    - comfyui
    - document-runtime
    - voice-provider
  policies:
    - content-provenance
    - commercial-licensing
    - child-safety
  apps:
    - course-builder
    - education-admin
```

10.1 Future Examples

- REXO Software Factory: architecture, implementation, QA, DevOps and observability.
- REXO Creative: image, video, voice, animation, design and publishing.
- REXO Marketing: research, strategy, campaigns, content, media and analytics.
- REXO GameDev: design, narrative, art, code, audio, testing and distribution.
- REXO Enterprise: internal integrations, compliance, private data and automations.

REFERENCE DISTRIBUTION

11. REXO Education

REXO Education will be the first vertical and the architecture's validation lab. It exercises research, planning, documents, code, media, QA, publishing and analytics without introducing educational rules into the core.

11.1 Composition

Component	Scope
Education Domain Pack	Pedagogy, curriculum, rubrics, taxonomy, policies and learning analytics
Course Production Pack	Research, objectives, modules, lessons, scripts, assessments and packaging
Creative Media Packs	Slides, images, diagrams, voice, animation, video and thumbnails
Publishing Pack	SEO, landing pages, LMS, video, files and metadata
Course Builder App	Creation, tracking, previews, approvals and publishing
Education Admin App	Catalog, costs, performance, quality, students and evolution

11.2 Full Pipeline

1. Discovery and audience definition.

2. Research with sources and technical validation.
3. Learning objectives and curriculum design.
4. Modules, lessons, dependencies and workload estimation.
5. Scripts, examples, demonstrations and storyboards.
6. Slides, diagrams, PDFs, images, voice, animation and video.
7. Exercises, quizzes, projects, answer keys and rubrics.
8. Technical, pedagogical, audiovisual, legal and accessibility QA.
9. Landing page, SEO, thumbnails, Shorts, Reels and promotional materials.
10. Publishing and packaging for configured destinations.
11. Production and learning analytics.
12. Evidence-based curriculum improvement proposals.

11.3 Educational Quality Gates

- Coverage of learning objectives.
- Technical correctness and currency.
- Difficulty progression and prerequisites.
- Clarity, examples and cognitive load.
- Alignment between lesson, exercise and assessment.
- Readability, audio, synchronization, accessibility and usage rights.
- Provenance of sources and assets.

PART

Quality, Economy and Governance

Quality Engine, security, reproducibility, Economy Engine and controlled evolution.

04



QUALITY ENGINE

12. Quality and Evaluation

There will be no universal score like 9.5. Each artifact type and risk class has a Quality Profile with mandatory validators, thresholds, evaluators, sampling and approval rules.

Layer	Use
Deterministic	Schema, compilation, lint, tests, hashes, duration, format
Model-based	Rubrics for clarity, coherence, aesthetics and completeness
Comparative	Select the best option among candidates
Independent review	Reviewer without access to the producer's rationale when appropriate
Human gate	High risk, publishing, cost, rights, medical, legal and financial matters
Regression	Compare new version against baseline and golden datasets
Production feedback	Real errors, satisfaction, performance and learning analytics

12.1 Definition of Done

- Output satisfies schema and acceptance criteria.
- Mandatory tests have passed.
- Sources and provenance are present.
- There are no policy violations.
- Costs and limitations have been recorded.
- Artifacts have been versioned and have hashes.
- Required approvals have been obtained.
- The final package contains a quality report.

TRUST

13. Security, Isolation and Governance

13.1 Trust Boundaries

- User and tenant.
- Project.
- Run.
- Worker and sandbox.
- Pack and publisher.
- Provider and external tool.
- Memory scope.
- Artifact visibility.

13.2 Mandatory Rules

Area	Rule
Credentials	Never enter prompts, logs, artifacts or memories; they are injected at the gateway.
Permissions	Granted via capability token, task, scope and time.
Data	Classified, encrypted, retained and deleted per policy.
Plugins	Signed, analyzed, permissioned and isolated.
Audit	Relevant events are immutable and correlated by trace.
Critical actions	Require explicit gates and record of the approver.
Models	Receive only the minimal necessary context.
Supply chain	Dependencies and artifacts have SBOM and hashes.

13.3 Human-in-the-loop

Autonomy is configurable by domain, risk and tenant. The system must distinguish clarifying questions, plan approvals, external-effect authorizations and specialist reviews.

LIFECYCLE

14. Versioning and Reproducibility

- SemVer: Agent Definitions, Skills, capabilities, workflows, packs, providers, policies and evaluators.
- Schema versioning: explicit compatibility and migrations.
- Lockfiles: complete resolution of a distribution and of a run.
- Immutable artifacts: published outputs are never overwritten.
- Content hashes: integrity and deduplication.
- Release channels: development, canary and stable.
- ADRs: architectural decisions and their consequences.
- RFCs: broad changes, contracts and ecosystem extensions.

14.1 Run Manifest

Every execution records versions of the Core, distribution, packs, Skills, Agent Definitions, workflows, policies, models, prompts, providers, tools, input data, context manifest, outputs, evaluators, costs and events. This enables auditing and reproduction proportional to the non-deterministic nature of the models.

OPERATIONS

15. Economy Engine, Observability and Operations

15.1 Economy Engine

Economy Engine — decision sequence



The Economy Engine is the component that decides, before any paid or probabilistic call, whether it should actually happen. It cuts across the Control Plane and the Scheduler and applies to every operation that consumes tokens, model calls, paid providers or significant compute time.

Reuse before generating. Generate only what is necessary. Never below the quality floor.

15.2 Economic Decision Sequence

1. Is there a valid, already-produced artifact with a compatible fingerprint? Reuse it instead of recreating it.
2. Is an incremental rebuild possible, recomputing only the part that changed?
3. What is the minimum context needed for the task, without sending the entire repository or history?
4. What is the smallest provider or model capable of meeting the required quality profile?
5. How much budget remains in tokens, calls, time and cost for this run, workflow and task?
6. What is the expected quality of the result, and what is the fallback if it is not reached?

15.3 Economic Rules

- Cost optimization can never cross the minimum quality floor declared for the capability.
- Ephemeral workers receive only the context necessary for the task; broad context is a justified exception, not the default.
- Cache keys include all relevant policy inputs, to prevent unsafe reuse of artifacts.
- Every reuse, incremental rebuild, or provider downgrade decision is recorded in the run's Resolution Record.
- Competition between providers only occurs when the expected value of the result justifies the additional cost.

15.4 Telemetry

- Traces by goal, run, task, capability, provider and evaluator.
- Structured logs without secrets or improper data.
- Quality, cost, latency, retries, failure and cache metrics.
- Lineage across inputs, decisions, artifacts and publications.
- Operational dashboards and SLO-based alerts.

15.5 Budget Manager

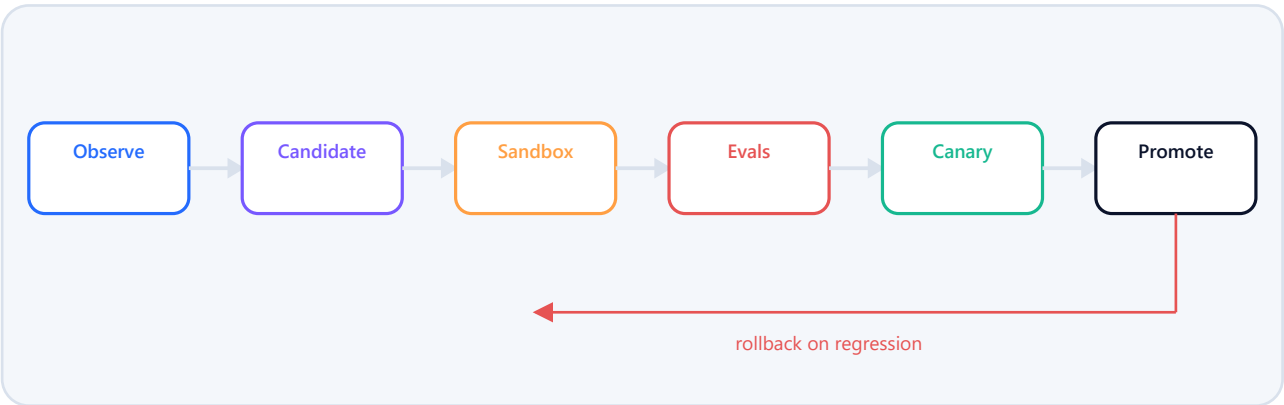
Budgets will be hierarchical: tenant, project, run, workflow and task. The scheduler reserves, consumes and returns budget. Deviations, competition between providers and regenerations require explicit limits.

15.6 Initial SLOs

Dimension	Indicators
Reliability	Completion, recovery, idempotency and failure rate
Quality	Approval, regression, correction and satisfaction
Performance	Queue, execution, provider and total time
Cost	Cost per task, artifact, workflow and delivery
Security	Violations, denied access, packages and secrets
Knowledge	Freshness, provenance, reuse and contamination

LEARNING ENGINE

16. Continuous Evolution



Self-evolution means producing and testing improvement candidates. It does not mean allowing the system to directly alter its own foundation.

16.1 Improvements That Can Be Proposed

- Skills, templates, prompts, rubrics and examples.
- Resolution heuristics and model routing.
- New agent compositions.
- Workflow optimizations.
- Evaluators and golden datasets.
- New providers and capability implementations.
- Documentation and curated knowledge.

16.2 Protected Components

- Core security and authorization policies.
- The audit mechanism.

- The promotion pipeline.
- Credentials and trust roots.
- Core's fundamental contracts.
- Mandatory human gates.

16.3 Promotion

Candidates are built in a sandbox, compared against a baseline, subjected to regression, security and cost checks, released as a canary, observed, and then promoted or rolled back.



PART

Structure and Execution

Repository layout, contracts, risks, roadmap and the operating model.

05



MODULAR MONOREPO

17. Directory Structure

The repository contains versioned contracts and components. Operational data, memories, artifacts, secrets and caches remain outside of Git.

```

rexo/
|-- README.md
|-- ARCHITECTURE.md
|-- SECURITY.md
|-- GOVERNANCE.md
|-- CONTRIBUTING.md
|-- VERSION
|-- docs/
|   |-- vision/
|   |-- architecture/
|   |-- adr/
|   |-- rfc/
|   |-- governance/
|   |-- runbooks/
|   |-- threat-models/
|-- contracts/
|   |-- capability/
|   |-- agent/
|   |-- skill/
|   |-- workflow/
|   |-- task/
|   |-- event/
|   |-- artifact/
|   |-- memory/
|   |-- evaluation/
|   |-- tool/
|   |-- package/
|-- platform/
|   |-- control-plane/
|   |   |-- intent-gateway/
|   |   |-- goal-compiler/
|   |   |-- planner/
|   |   |-- plan-validator/
|   |   |-- orchestrator/
|   |   |-- scheduler/
|   |   |-- registry/
|   |   |-- policy-engine/
|   |-- execution-plane/
|   |   |-- worker-runtime/
|   |   |-- sandbox-manager/
|   |   |-- context-service/
|   |   |-- artifact-service/
|   |-- knowledge-plane/
|   |   |-- memory-service/
|   |   |-- knowledge-service/
|   |   |-- ingestion-service/
|   |   |-- provenance-service/
|   |   |-- retrieval-service/
|   |-- quality-plane/
|   |   |-- evaluation-engine/
|   |   |-- review-engine/
|   |   |-- regression-runner/
|   |   |-- approval-service/
|   |-- capability-fabric/
|   |-- tool-gateway/
|   |-- package-manager/
|   |-- eventing/
|   |-- observability/
|   |-- security/
|   |-- cost-management/
|-- components/
|   |-- agents/<domain>/<name>/
|   |-- skills/<domain>/<name>/
|   |-- workflows/<domain>/<name>/

```

```
| |-- evaluators/<domain>/<name>/
| |-- templates/<domain>/<name>/
| |-- providers/<provider>/
| `-- packages/<package>/
|-- distributions/
|   |-- education/
|-- policies/
|-- sdk/
|   |-- capability-sdk/
|   |-- agent-sdk/
|   |-- skill-sdk/
|   |-- workflow-sdk/
|   |-- package-sdk/
|   |-- evaluator-sdk/
|   `-- testing-sdk/
|-- catalog/
|   |-- taxonomy/
|   |-- compatibility/
|   `-- generated/
|-- evals/
|-- tests/
|-- apps/
|   |-- api/
|   |-- cli/
|   |-- admin-console/
|   |-- operator-console/
|   `-- marketplace/
|-- infrastructure/
`-- examples/
```

17.1 Generated Catalog

Component manifests are the official source. The generated catalog directory is a rebuildable index and must not manually duplicate definitions.

EXTENSIBILITY

18. Fundamental Contracts

Contract	Function
Goal Specification	Goal, deliverables, constraints, budgets, risks and acceptance criteria
Plan DAG	Tasks, dependencies, gates, retries, timeouts and outputs
Task Envelope	Execution contract between orchestrator and worker
Capability Specification	Abstract outcome and its schemas
Capability Implementation	Concrete provider and its constraints
Artifact Manifest	Type, hash, version, lineage, sources and status
Memory Record	Scope, validity, confidence, provenance and retention
Evaluation Report	Criteria, evidence, scores, failures and decision
Package Manifest	Content, dependencies, permissions and compatibility
Run Manifest	Complete resolution and execution history
Event Envelope	Versioned, idempotent and correlated event

18.1 Extension Without Core Changes

New agents, Skills, workflows, providers, evaluators, packs and Apps are discovered through compatible manifests. The Core does not receive domain-specific conditionals. Core changes are only justified when a general contract is insufficient.

RISK REGISTER

19. Risks and Controls

Risk	Impact	Control
Task explosion	Fan-out and rising costs	Budgets, depth limits, task merging and backpressure
Progress-less loops	Infinite retries and reviews	Limits, progress detector and escalation
Memory poisoning	Persistent error across projects	Provenance, curation, scope and promotion
Skill duplication	Confusing, inconsistent catalog	Taxonomy, overlap detection, ownership and deprecation
Supply chain	Malicious or vulnerable packages	Signing, SBOM, sandbox, trust tiers and scanning
Provider lock-in	Workflows tied to a vendor	Capability contracts and adapters
Quality gaming	Good metric, bad result	Varied evals, independent review and real feedback
Knowledge staleness	Outdated content	Freshness, TTL, revalidation and authoritative sources
Cost opacity	Parallelism and regeneration	Budget manager, quotas and estimation
Overengineering	Infrastructure ahead of product	Modular monolith and first vertical
Data leakage	Context crossing tenants	Isolation, least privilege and context manifest
Model churn	Frequent model changes	Resolver, compatibility matrix and regression suite

ROADMAP

20. Future Implementation Strategy

The ambition is broad; the initial implementation must be narrow. The rule will be a universal architecture with a first vertical distribution.

Phase	Outcome
Phase 0 - Constitution	Approve principles, contracts, ADRs, taxonomy and threat model.
Phase 1 - Minimal kernel	Runs, tasks, durable orchestration, registry, worker runtime, artifacts and observability.
Phase 2 - Capability Fabric	Specifications, implementations, resolver, policies, Tool Gateway and lockfiles.
Phase 3 - Memory and Quality	Context Service, Project Memory, provenance, evaluators and quality gates.
Phase 4 - Education MVP	Research, curriculum, lessons, scripts, documents and QA; media initially controlled.
Phase 5 - Creative providers	ComfyUI, voice, video, diagrams, thumbnails and comparative evaluation.
Phase 6 - Package Manager	Installable packs, dependencies, permissions, migrations and rollback.
Phase 7 - REXO Education	Course Builder, publishing, analytics and end-to-end operation.

Phase	Outcome
Phase 8 - Private marketplace	Publisher workflow, trust tiers, signing, review and internal distribution.
Phase 9 - Platform factory	Distribution Builder, SDKs and new distributions.
Phase 10 - Public marketplace	Only after sandbox, security and governance maturity.

20.1 Criteria for Extracting Microservices

Components will only be physically separated when there is proven demand for scale, isolation, security, ownership or deploy cycle. Logical modularity comes before physical distribution.

OPERATING MODEL

21. How We Will Work

21.1 Decision Process

1. Define the problem, context, invariants and alternatives.
2. Record relevant decisions in an ADR.
3. Use an RFC for changes affecting contracts, the ecosystem or security.
4. Build the smallest increment that tests the architectural hypothesis.
5. Create evals and criteria before or alongside implementation.
6. Measure quality, cost, latency, security and operability.
7. Promote only after evidence; roll back when there is regression.

21.2 Rules for the Chief AI Architect

- Critique ideas, including the user's own, when there is risk or a superior solution.
- Avoid premature irreversible decisions.
- Preserve separation of responsibilities and contracts.
- Do not create abstractions without at least one real case and a second plausible case.
- Do not turn a provider preference into an architectural decision.
- Require security, observability, migration and rollback in extensible features.
- Treat knowledge and quality as products, not accessories.
- Keep documentation and ADRs synchronized with the system.

21.3 Rules for Future Deliveries

- No component is complete without tests and evals proportional to risk.
- No capability is published without schemas and a reference implementation.
- No pack is installed without a permission diff and compatibility check.
- No critical external action occurs without corresponding authorization.
- No automatic improvement reaches stable without regression and canary.
- No organizational knowledge is promoted without provenance.

BASELINE

22. Architectural Decisions Approved in This Proposal

ID	Decision	Rationale
AD-001	REXO, not AI Company	The agent company is one possible experience; the product is the extensible runtime.
AD-002	Capability-first	Capabilities are the central contracts; agents are optional compositions.
AD-003	Stable core and installable domains	Education and other verticals do not alter the core.
AD-004	Durable orchestration	Long-running workflows are persistent, resumable and auditable.
AD-005	Central Tool Gateway	MCPs and tools are not accessed directly by agents.
AD-006	Layered memory	No unrestricted global memory; context is assembled by policy.
AD-007	Executable quality	Evals and gates replace generic scores.
AD-008	Controlled evolution	Self-evolution goes through promotion and rollback.
AD-009	Modular monolith first	Physical distribution only upon proven need.
AD-010	Education as reference distribution	The first product validates the Core without contaminating it.
AD-011	Typed package ecosystem	Packs have types, contracts, permissions and a lifecycle.
AD-012	Platform for platforms	The Distribution Builder composes specialized platforms.
AD-013	Developer & Creation Plane	SDK, CLI, Studio, Canvas and Creator form an official layer.
AD-014	One contract model	All surfaces produce the same manifests and schemas.
AD-015	Studio contains Canvas	A unified visual interface avoids parallel products and formats.
AD-016	Runtime-aware marketplace	Packs can resolve signed, isolated execution environments.
AD-017	Creator is not privileged	The Platform Factory uses the SDK and does not publish directly to stable.
AD-018	Explicit operating modes	Architecture, engineering and operation transitions are declared and gated.

Final state: this version defines the baseline architecture for discussion and approval. It does not authorize automatic implementation. After approval, Phase 0 must turn these decisions into ADRs, contracts and verifiable criteria.

PART

Developer Experience

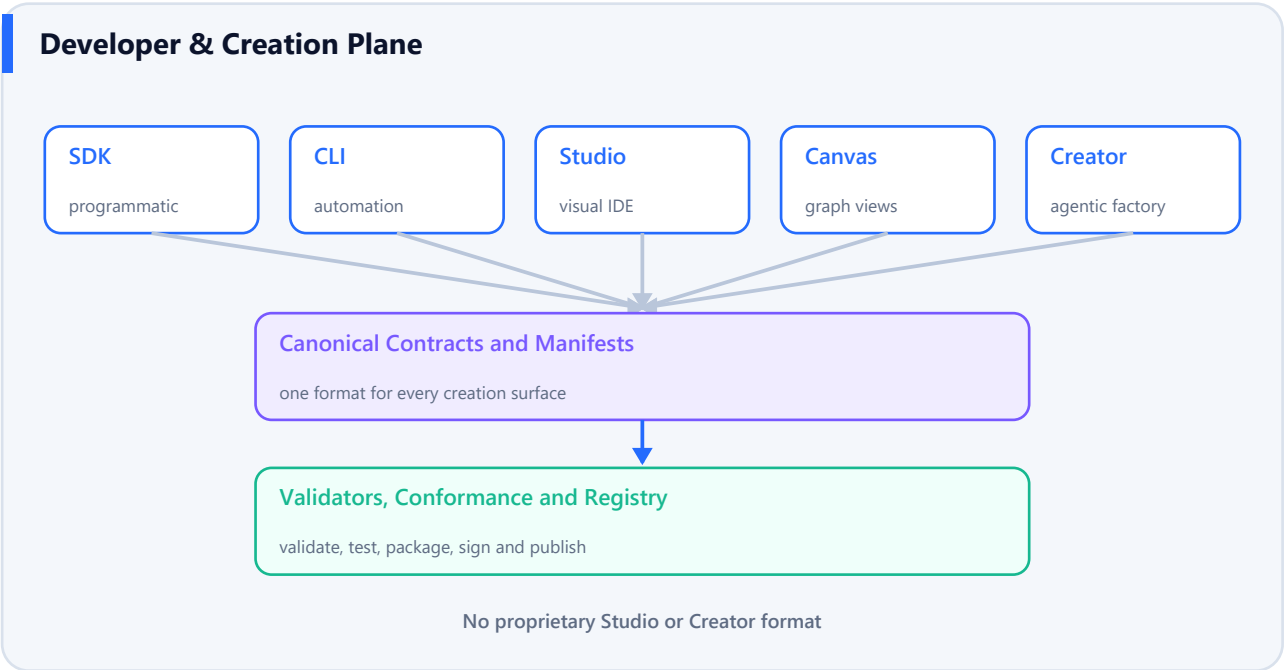
SDK, CLI, Studio, Canvas, marketplace, Creator and the operating modes.

06



DEVELOPER EXPERIENCE

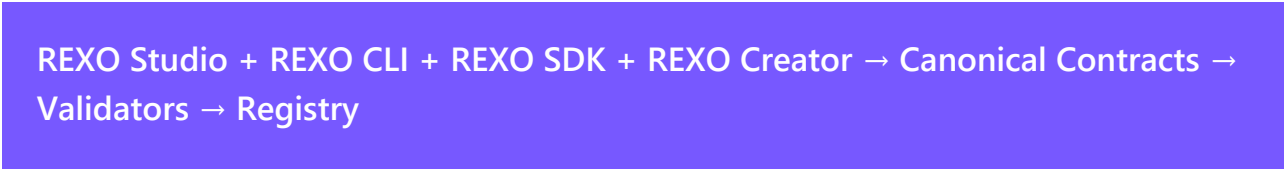
23. Developer & Creation Plane



Architecture only becomes an ecosystem when correct extensions can be created quickly. The Developer & Creation Plane is the layer that turns internal contracts into product journeys for developers, operators and creators.

23.1 Single-Format Invariant

SDK, CLI, Studio, Canvas and Creator must read and write the same manifests, schemas, workflows, lockfiles, policies and evaluation profiles. There will be no proprietary visual format nor a secret API for the Creator.



23.2 Components

Component	Responsibility
REXO SDK	APIs, builders, clients, testing and conformance for extensions
REXO CLI	Scaffolding, validation, testing, packaging, installation and publishing
Generators	Consistent starting structures for each component type
Validators	Schemas, semantics, permissions, compatibility and quality baselines
REXO Studio	Visual IDE, operational console, debugger and governance workspace
REXO Canvas	Graphical views of workflows, capabilities and domain entities
REXO Creator	Agentic Platform Factory for generating ecosystem candidates

23.3 Extensibility Rule

An extension is only valid when it can be created via file, SDK, CLI or Studio and produce an equivalent canonical representation. Any feature available in only one surface will be considered architectural debt.

CONSTRUCTION KIT

24. REXO SDK

The SDK is the ecosystem's official construction kit. It will not be just an HTTP client: it will provide contracts, builders, runtime clients, mocks, a local harness, validators, test fixtures, conformance suites and packaging tools.

```
rexo-sdk/  
|-- contracts/  
|-- manifest-builders/  
|-- capability-api/  
|-- provider-api/  
|-- workflow-dsl/  
|-- agent-composition/  
|-- memory-api/  
|-- policy-api/  
|-- evaluation-api/  
|-- package-api/  
|-- distribution-api/  
|-- local-harness/  
|-- mocks/  
|-- validators/  
`-- conformance-kit/
```

24.1 Conceptual APIs

Builder	Result
createCapability()	Declares the abstract contract, schemas, effects, risk and quality profile
createProvider()	Registers a concrete implementation and health/cost metadata
createSkill()	Packages competence, resources, tests and evals
createWorkflow()	Composes a durable DAG, gates, retries and migrations
createPack()	Groups components, dependencies, permissions and lifecycle
createApp()	Creates an experience that consumes capabilities
createDistribution()	Composes packs, providers, policies and Apps
createEvaluator()	Defines an executable evaluation and baseline

24.2 Language Strategy

Schemas and manifests will be language-neutral. SDKs will be generated from the canonical contracts. The choice of the first languages will be made via ADR, but the architecture assumes at least one strong experience for automation and another for applications.

24.3 Conformance Test Kit

- Validates schemas, ids, versions, dependencies and capability semantics.
- Runs examples and golden tests.
- Tests idempotency, retries, cancellation and timeout.
- Simulates provider unavailability, partial failure and rollback.
- Verifies the permissions manifest and the absence of excess access.
- Produces a report publishable on the marketplace.

DEVELOPER WORKFLOW

25. REXO CLI, Scaffolding and Generators

The CLI will be the first development and automation surface. The Studio will consume the same services and contracts, avoiding divergence between the visual experience and code.

25.1 Target Interface

The commands below define the desired future experience. They do not yet exist at this architectural stage.

```
rexo create capability media.generate-video
rexo create provider specialist-video-provider
rexo create workflow course-production
rexo create pack education-assessments
rexo create distribution legal-rexo

rexo validate    rexo test    rexo eval
rexo pack       rexo install education    rexo publish
```

25.2 Pack Creation Journey

1. Select the Pack type and namespace.
2. Generate scaffolding with manifest, schemas, examples, tests and evals.
3. Add provided capabilities and dependencies.
4. Declare permissions, data policy and runtime requirements.
5. Run static and semantic validation.
6. Run local tests in a sandbox.
7. Run the conformance and regression suite.
8. Generate an immutable package, SBOM, signature and changelog.
9. Publish to the development or canary channel.
10. Promote to stable after trust criteria are met.

25.3 Generators

Generators provide opinionated starting structures but do not hide contracts. Templates are versioned, replaceable and tested. Generator updates do not silently modify existing components.

VISUAL PRODUCT

26. REXO Studio and REXO Canvas

REXO Studio will be a unified visual interface inspired by IDEs and creative engines. REXO Canvas will be the set of graphical views within the Studio, not a separate product.

Studio Mode	Capabilities
Design Mode	Workflows, Packs, capabilities, providers, templates, policies and distributions
Run Mode	Live execution, pause, resume, approvals, artifacts and costs
Debug Mode	Task envelopes, context, resolution, retries, fallbacks and replay
Governance Mode	Permissions, packages, trust, audit, retention and release
Marketplace Mode	Discovery, comparison, install plans, updates and rollback

26.1 Workflow Canvas

Users connect capabilities visually. Each edge represents an artifact or event contract. The editor prevents incompatible connections, requires gates for critical effects, and generates the same Workflow Definition used by the CLI.

Research → Curriculum → Slides → Video → QA → Publish

26.2 Domain Canvas

The Domain Canvas represents entities and artifacts, not just tasks. For Education, the user manipulates Course, Module, Lesson, Video, PDF, Image, Quiz and Exercise.

- Reorder, duplicate and create dependencies.
- Lock approved artifacts against regeneration.
- Swap the provider for a single node only.
- Compare versions and candidates.
- Re-run only a subtree.
- View provenance, cost, quality and status.
- Publish a consistent version of the set.

26.3 Round-trip Fidelity

Opening a workflow created by the CLI in the Studio and saving it must not change its semantics. Visual changes must produce readable, reviewable diffs.

DISTRIBUTION

27. Runtime Marketplace and Installation Lifecycle

The ecosystem will have three coordinated catalogs: Package Marketplace, Runtime Catalog and Distribution Marketplace.

Catalog	Distributes
Package Marketplace	Packs, Skills, workflows, Apps, templates, evaluators, policies and providers
Runtime Catalog	Worker profiles, sandboxes, environments, native dependencies and GPU profiles
Distribution Marketplace	Complete, composed and versioned platforms

27.1 Transactional Installation

1. Resolve distribution, packs, providers, runtimes and policies.
2. Verify signature, publisher, trust tier, SBOM and vulnerabilities.
3. Compute permission diff, hardware requirements, licenses and cost.
4. Detect conflicts and build the install plan.
5. Request approval when permissions or effects require it.
6. Download and verify artifacts by hash.
7. Install in staging and run reversible migrations.
8. Run smoke tests and conformance checks.
9. Activate atomically and write the lockfile.
10. Keep a rollback plan and the previous version when applicable.

27.2 Update and Removal

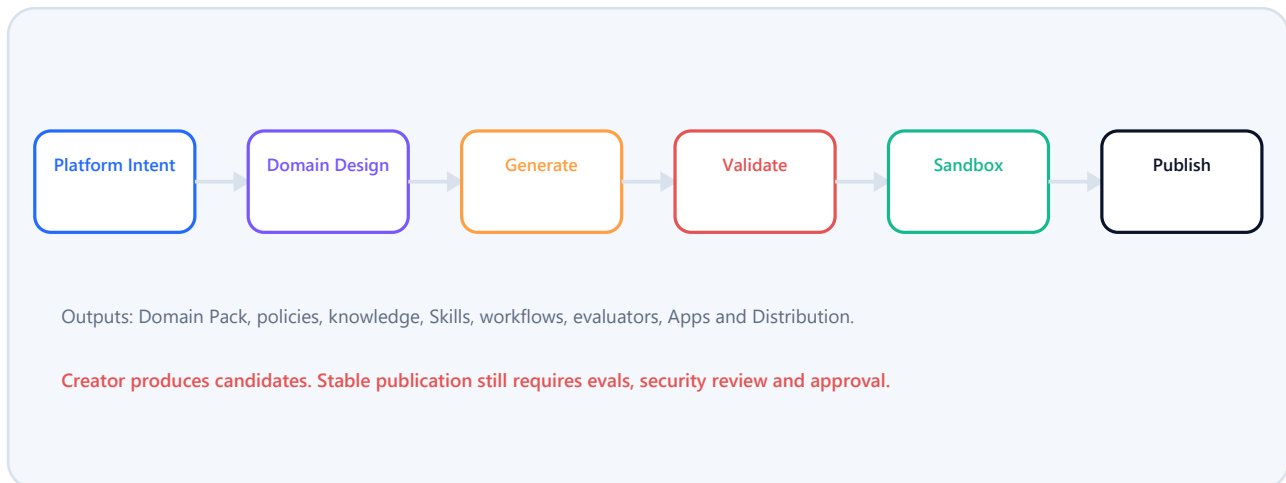
Updates produce a compatibility report, migration plan and permission diff. Removal checks dependencies, preserves data per policy, and runs the uninstall plan without deleting shared artifacts.

27.3 Runtime Profiles

Packs may require runtime capabilities, but must not run arbitrary installers on the host. Runtime profiles will be signed, reproducible, isolated and selected by the Scheduler.

PLATFORM FACTORY

28. REXO Creator



REXO Creator is an agentic application built on top of the SDK. It receives a Platform Intent Specification and produces a complete candidate for a specialized distribution.

28.1 Platform Intent Specification

- Domain, jurisdiction, audience and use cases.
- Risks, data, privacy and compliance.
- Expected capabilities and prohibited outcomes.
- Available providers and deployment constraints.
- Quality, budgets, autonomy and human gates.
- Desired Apps and experiences.

28.2 Outputs

- Domain Pack and taxonomy.
- Capability Packs and Provider requirements.
- Policies, threat model and data classification.
- Knowledge ingestion pipelines.
- Skills, workflows, templates and evaluators.
- Apps and initial interfaces.
- Distribution manifest, development lockfile and documentation.
- Evals, golden datasets and evolution plan.

28.3 Limits

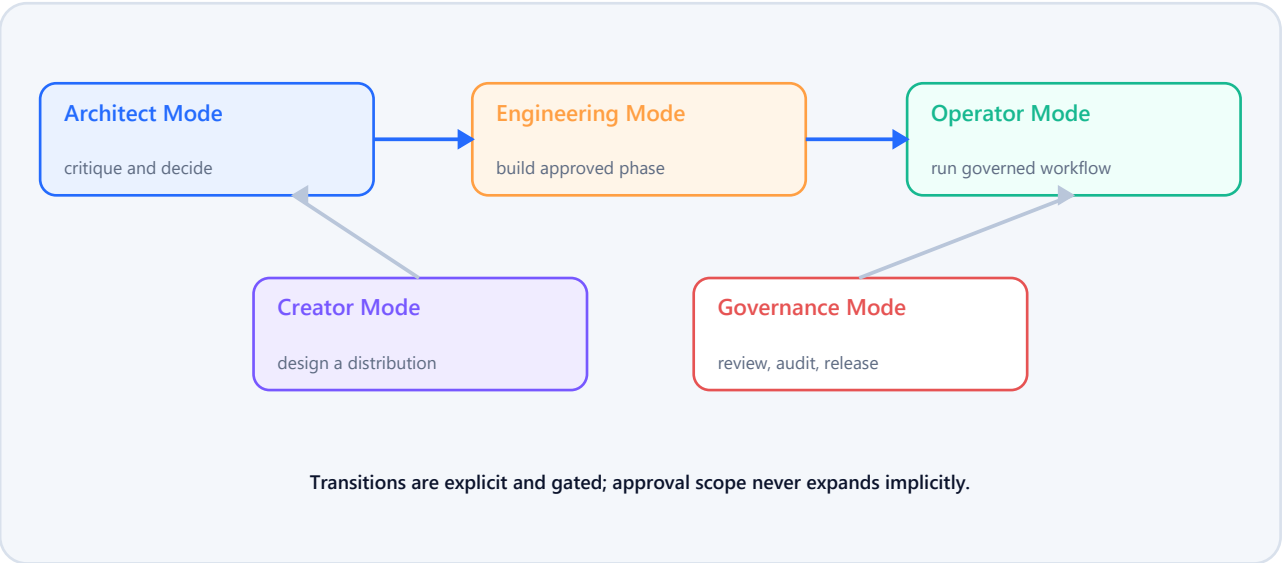
Creator does not modify the Core, does not weaken mandatory policies, does not publish directly to stable, and does not grant itself new permissions. Its outputs go through validators, security review, evals, sandbox, canary and promotion.

28.4 Example

Input: a system specialized in Brazilian legal practice for small law firms. Output: Legal Domain Pack, Brazilian Law Knowledge Pack, Legal Policy Pack, Contract Analysis Pack, research workflows, legal evaluators, a workspace App and the REXO Legal Distribution.

HOW REXO WORKS

29. Operating Modes and Activation



An operating mode is an explicit profile of authority, behavior, outputs and stop conditions. It is not a free-form personality, nor does it automatically increase permissions.

Mode	Behavior
Chief AI Architect	Critiques, compares alternatives, produces ADRs/RFCs and does not implement without approval
Ecosystem Designer	Defines contracts, SDK, Packs, providers, marketplace and Developer Experience
Product Engineering	Implements only the approved phase and scope, with tests and evidence
Operator	Runs published workflows under policies, budgets and gates
Creator	Generates candidates for new platforms and distributions
Governance	Audits, evaluates, approves, promotes, blocks or reverts releases

29.1 How to Activate With Me Right Now

Today, before the REXO runtime exists, the mode is a working contract within this conversation. It is activated by explicit natural-language instruction. There is no hidden button or installed command.

```
Architecture:
Activate Chief AI Architect mode for this project. Critique the proposal, produce decisions, and do not
implement any files.

Engineering:
I approve architecture v2. Activate Product Engineering only for Phase 0, following the gates in the
document.

Creator:
Activate REXO Creator to design a legal distribution. Generate only the candidate specification and the
risks.

Operation:
Activate Operator Mode to run workflow X with budget Y and approval before publishing.
```

29.2 Activation Confirmation

When a mode is activated, the system must respond with a Mode Activation Record containing: mode, goal, scope, authority, restrictions, expected artifacts, budgets, gates, stop conditions and initial state.

29.3 Future CLI Interface

The commands below are part of the target specification and are not currently implemented.

```
rexo mode activate architect --project .
rexo mode activate engineering --phase 0
rexo mode activate creator --intent legal-platform.yaml
rexo mode status
rexo mode transition engineering --approval ADR-SET-001
rexo mode deactivate
```

29.4 State Machine

DRAFT → ARCHITECTURE_REVIEW → ARCHITECTURE_APPROVED → PHASE_PLANNED → BUILDING → VALIDATING
→ CANARY → STABLE

- Transitions require evidence and approval as defined by policy.
- Approving the architecture does not authorize all implementation phases.
- Approving a phase does not authorize external publication.
- A new material objective creates a new activation or amendment.
- A mode can be paused, revoked or reduced at any time.
- Severe failures transition to BLOCKED, ROLLBACK or INCIDENT.

29.5 How I Will Operate on the Project

Moment	Obligation
Before acting	Read state, mode, contracts, ADRs, policies, budget and relevant artifacts
When planning	Produce tasks, dependencies, risks, criteria and verifiable gates
When deciding	Explain alternatives and record material architectural decisions
When executing	Use only authorized tools, data and scope
When reviewing	Apply independent evals and fix specific failures
When learning	Record candidates; do not automatically promote knowledge
When concluding	Deliver artifacts, evidence, limitations, costs and the next state



PART

Appendices

Glossary, approval checklist and the next authorised step.

07



REFERENCE

Appendix A. Glossary

Term	Definition
REXO Core	The stable core of the runtime and platform services.
Capability	An abstract outcome that can have multiple implementations.
Implementation	A concrete way of satisfying a capability.
Provider	An integration with a model, service, MCP, API, CLI or local tool.
Skill	A versioned unit of reusable competence.
Agent Definition	A logical role composed of Skills, policies and decision-making capabilities.
Worker	An ephemeral instance that executes a task.
Workflow	A durable DAG of tasks, gates, evaluations and compensations.
Pack	An installable, versioned unit of the ecosystem.
App	A user-facing experience built on top of capabilities.
Distribution	A composition of Core, packs, providers, policies and apps.
Artifact	A versioned, traceable output produced by the system.
Run	The execution of a complete goal.
Task	A scheduled unit of work within a run.
Eval	An executable evaluation of quality, security or performance.
Policy	A rule enforced by the system independently of the prompt.
Provenance	The history of origin, transformation and authorship of data and artifacts.
REXO SDK	The official kit for building, testing and packaging extensions.
REXO Studio	A unified visual interface for design, operation, debugging and governance.
REXO Canvas	The set of graphical views within the Studio.
REXO Creator	The agentic Platform Factory that generates candidate distributions.
Operating Mode	An explicit profile of authority, behavior and stop conditions.

NEXT STEP

Appendix B. Approval Checklist

- ☐ Does the definition of REXO correctly represent the intended product?
- ☐ Is capability-first accepted as the central principle?
- ☐ Are agents accepted as optional compositions, not the fundamental unit?
- ☐ Is the separation of Core, Packs, Apps and Distributions approved?
- ☐ Is REXO Education approved as the first vertical?

- □ Does the public marketplace remain contingent on security maturity?
- □ Is the modular monolith accepted as the initial topology?
- □ Are governed memory and controlled promotion approved?
- □ Are the central Tool Gateway and capability tokens approved?
- □ Does the phase sequence fit the project's priorities?
- □ Is the Developer & Creation Plane accepted as an official layer?
- □ Must SDK, CLI, Studio and Creator produce the same canonical format?
- □ Must REXO Canvas remain a view within the Studio?
- □ Are the Runtime Marketplace and transactional installation approved?
- □ Are the operating modes and their activation protocol approved?

CLOSING

Next Step

Approval of this Constitution does not automatically start implementation. It authorizes Phase 0, whose goal is to convert the vision into contracts, decisions and verifiable criteria.

Phase 0 Deliverable	Expected Outcome
ADRs	Individual architectural decisions, alternatives and consequences
Contracts	Initial schemas for Goal, Task, Capability, Artifact, Eval, Event and Package
Threat model	Trust boundaries, risks, controls and mandatory gates
Taxonomy	Official capability namespaces and conventions
DX Specification	SDK, CLI, scaffolding, Studio, Canvas, marketplace and Creator
Executable roadmap	Epics, dependencies, completion criteria and build order

Universal architecture. First vertical implementation. Evidence-driven evolution.